

Extended Abstract

Motivation Enhanced Geothermal Systems (EGS) represent a promising source of clean baseload energy, but drilling through hard crystalline rock accounts for 30–57% of total project cost. Conventional drilling optimization relies on physics-based rate-of-penetration (ROP) models and greedy one-step control, which ignore temporal tradeoffs such as bit wear accumulation. This work investigates whether deep reinforcement learning can learn policies that optimize drilling parameters—weight-on-bit (WOB), rotation speed (RPM), and mud flow rate—to maximize ROP while avoiding mechanical dysfunctions like stick-slip vibration.

Method We developed an open-source Gymnasium-compatible environment that simulates EGS drilling using a Bourgoyne-Young ROP model calibrated on Utah FORGE well data. The environment features an 8-dimensional continuous state space (depth, ROP, torque, WOB, RPM, flow, bit wear, and rolling mean ROP), a 3-dimensional continuous action space (deltas to WOB, RPM, and flow), and a shaped reward function: $R = \text{ROP} - \lambda_1 \cdot \text{torque} - \lambda_2 \cdot |\Delta \text{action}| - \lambda_3 \cdot \text{dysfunction}$. We trained Proximal Policy Optimization (PPO) and Deep Deterministic Policy Gradient (DDPG) agents for 200,000 timesteps across 3 random seeds each, comparing against four baselines: constant control, rule-based heuristics, behavioral cloning, and a grid-search oracle.

Implementation The Bourgoyne-Young model was calibrated on FORGE well 16A (78-32) with 29,897 granite drilling rows, achieving $R^2 = 0.30$ on the training well but $R^2 = -52.7$ on holdout well 58-32, indicating large overfitting. The environment includes stochastic hard rock stringers (5% probability per 100 ft) and a formation transition from sedimentary to granite at 8,000 ft depth. Training used Stable Baselines3 with default hyperparameters. For sim-to-sim validation, we deployed trained policies on NORCE’s OpenLab, an independent high-fidelity drilling simulator used by industry.

Results When training converges (seed 0), PPO achieved the highest reward (8450) among all methods, outperforming Constant (7282) and Grid-Oracle (6519). However, both RL algorithms exhibited severe training instability: 2 of 3 seeds collapsed for each algorithm. Averaging across seeds yields PPO: 2820 ± 3981 and DDPG: 1081 ± 1527 —standard deviations exceeding means. All four hand-designed baselines were stable across seeds. Ablation on the dysfunction penalty λ_3 revealed a threshold effect: values 0–1.0 produced identical 100% dysfunction rates, while $\lambda_3 = 2.0$ reduced dysfunction to 65% but drove reward negative. OpenLab validation revealed divergent behaviors: PPO achieved 3.2 m/hr with 92% dysfunction (aggressive), while DDPG achieved 1.2 m/hr with 0.5% dysfunction (conservative).

Discussion The two RL algorithms converged to opposite extremes of the speed-safety tradeoff, with neither finding a balanced policy. PPO’s on-policy updates may have reinforced early high-ROP experiences, while DDPG’s replay buffer may have retained dysfunction penalties more strongly. The Grid-Oracle baseline achieved 87% dysfunction despite perfect one-step lookahead, confirming that greedy control is insufficient and multi-step reasoning is required.

Conclusion We present the first open-source RL environment for geothermal drilling and provide evidence that deep RL is fragile in this domain. Key contributions include: (1) DrillingEnv with EGS-specific physics, (2) documentation of 2/3 seed collapse for both PPO and DDPG, indicating possible RL instability in these realms (3) quantification of divergent behaviors via OpenLab validation, and (4) evidence that algorithm choice significantly affects learned tradeoffs. Future work should address: (1) improved surrogate calibration using additional variables (differential pressure, bit RPM, mud properties), (2) multi-well training data to reduce overfitting, (3) reward normalization and extended training, and (4) alternative algorithms (SAC, TD3, model-based methods).

Reinforcement Learning for Geothermal Drilling Optimization

Devan Agrawal

Department of Computer Science
Stanford University
agrawald@stanford.edu

Abstract

Drilling through hard crystalline rock accounts for 30–57% of Enhanced Geothermal Systems (EGS) project cost. We investigate whether deep reinforcement learning can optimize drilling parameters to maximize rate of penetration while avoiding mechanical dysfunctions. We develop DrillingEnv, an open-source Gymnasium environment with physics-based dynamics calibrated on Utah FORGE well data. When training converges, PPO achieves the highest reward among all methods, outperforming hand-designed baselines—demonstrating that deep RL can learn effective drilling policies. However, both PPO and DDPG exhibit severe instability, with 2 of 3 training seeds collapsing for each algorithm. Validation on NORCE’s OpenLab reveals divergent behaviors: PPO learns aggressive drilling (3.2 m/hr, 92% dysfunction) while DDPG learns overly conservative control (1.2 m/hr, 0.5% dysfunction). These results show promise for RL in drilling optimization but highlight critical challenges in training stability and algorithm selection.

1 Introduction

1.1 Motivation

Enhanced Geothermal Systems (EGS) offer a path to clean, renewable baseload electricity by extracting heat from deep underground rock formations. Unlike solar and wind power, geothermal energy provides consistent output regardless of weather conditions, making it valuable for grid stability. However, EGS deployment remains limited, with drilling costs representing the primary economic barrier.

Drilling through hard crystalline rock—granite, gneiss, and other basement formations—accounts for 30–57% of total EGS project cost (1). Unlike conventional oil and gas drilling through softer sedimentary formations, geothermal drilling faces extreme conditions: temperatures exceeding 200°C, rock compressive strengths above 200 MPa, and abrasive minerals that rapidly wear drill bits. These challenges make drilling optimization particularly impactful: reducing drilling time directly reduces cost per kilowatt-hour of generated electricity.

1.2 Problem Formulation

The drilling optimization problem involves controlling three primary parameters in real-time:

- **Weight-on-bit (WOB):** The downward force applied to the drill bit, measured in pounds-force (lbf). Higher WOB increases cutting rate but also increases torque and bit wear.
- **Rotation speed (RPM):** The angular velocity of the drill string. Higher RPM increases cutting rate but generates more heat and vibration.

- **Mud flow rate:** The volume of drilling fluid pumped through the system, measured in gallons per minute (gpm). Mud cools the bit, carries rock cuttings to surface, and stabilizes the borehole.

The objective is to maximize rate of penetration (ROP)—how fast the drill advances through rock—while avoiding mechanical dysfunctions. The most common dysfunction is stick-slip vibration, where the drill string twists under torque, sticks momentarily, then releases suddenly. Stick-slip damages equipment, accelerates bit wear, and can cause costly drilling interruptions (2).

We formulate this as a Markov Decision Process (MDP) with continuous state and action spaces, shaped rewards, and episodic structure (one episode = drilling one well section).

1.3 Contributions

This work makes three contributions:

1. **DrillingEnv:** The first open-source Gymnasium-compatible RL environment for geothermal drilling, incorporating physics-based ROP modeling, stochastic formation heterogeneity, and realistic dysfunction mechanics.
2. **Empirical Analysis of Deep RL Stability:** We train PPO and DDPG and evaluate training stability compared against hand-designed baselines.
3. **Sim-to-Sim Validation:** We validate learned policies on NORCE’s OpenLab—an industry-grade simulator with physics absent from our surrogate—to test whether policies generalize and to assess surrogate fidelity.

2 Related Work

2.1 Physics-Based Drilling Models

The Bourgoyne-Young model (3) remains the foundation of drilling rate prediction after 50 years. The model expresses ROP as a function of WOB, RPM, bit wear, rock strength, and other factors through an empirical equation with tunable coefficients, simplified in a manner as expressed in the equation below:

$$\text{ROP} = A_1 \cdot \left(\frac{\text{WOB} - W_t}{10000} \right)^{A_2} \cdot \left(\frac{\text{RPM}}{60} \right)^{A_3} \cdot e^{-A_4 \cdot \text{wear}} \cdot f_{\text{formation}} \quad (1)$$

where A_1 – A_4 are tunable coefficients and W_t is the threshold WOB. While interpretable and grounded in drilling mechanics, the model has known limitations: coefficients must be calibrated per well, and the model does not capture complex dynamics like stick-slip vibration (4).

2.2 Drilling Automation and Control

Conventional drilling optimization relies primarily on rule-based control and operator expertise. Drillers monitor surface measurements and adjust parameters based on heuristics (e.g. reducing weight-on-bit when torque exceeds a threshold). While effective in practice, such approaches are inherently myopic and do not explicitly optimize long-term tradeoffs such as balancing immediate rate of penetration against future bit wear or vibration risk.

Model-based optimization and control approaches have also been explored in drilling (5). These methods leverage physics-based models of drill-string dynamics, but their performance depends heavily on model fidelity. Prior work has shown that drilling dynamics exhibit strong nonlinearities, distributed vibration modes, parameter sensitivity, and well-specific behavior, making accurate prediction and control challenging in real-world operations (5).

2.3 Reinforcement Learning for Drilling

Cayeux et al. (6) developed an autonomous drilling framework based on an MDP and validated it in NORCE’s OpenLab simulator. Their system selected drilling actions by balancing drilling performance against the risk of adverse events under significant uncertainty arising from sparse

downhole measurements. While the authors note reinforcement learning as a promising future direction, their work focused on autonomous decision-making for conventional oil and gas drilling. In contrast, geothermal drilling in hard crystalline formations presents substantially different rock mechanics, drilling dysfunctions, and operational objectives.

To our knowledge, no prior work has developed an open-source RL environment specifically for geothermal drilling, nor systematically studied the stability of deep RL algorithms in this domain. Our surrogate is configured for EGS conditions—hard crystalline rock (150–250 MPa UCS), formation transitions, and stochastic hard stringers—and calibrated on Utah FORGE well data, while remaining fast enough for RL training (thousands of episodes in minutes vs. hours for high-fidelity simulators like OpenLab).

3 Method

3.1 Environment Design

We implemented DrillingEnv as a Gymnasium-compatible environment simulating drilling from 6,500 to 10,000 feet depth. The environment models a well transitioning from sedimentary overburden (6,500–8,000 ft) to crystalline granite basement (8,000–10,000 ft), representative of EGS drilling at sites like the Utah FORGE project.

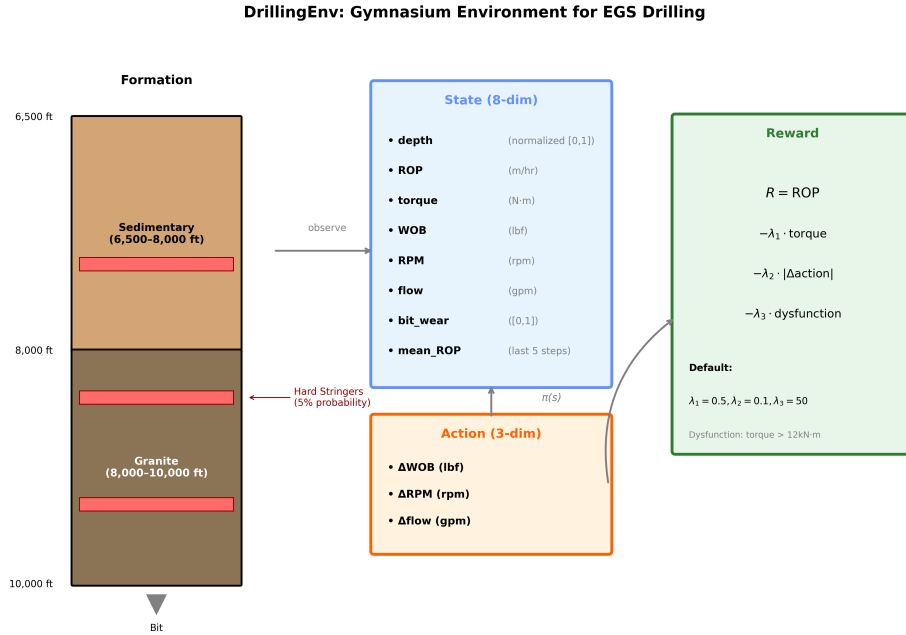


Figure 1: DrillingEnv architecture. The state space captures drilling conditions including bit wear (temporal state). Actions are delta changes to WOB, RPM, and flow. The reward balances ROP against torque penalties and dysfunction events. Formation transitions from sedimentary to granite at 8,000 ft.

3.1.1 State Space

The state space consists of 8 continuous variables, each chosen to provide information necessary for optimal control:

The inclusion of bit wear is critical: it captures the temporal tradeoff that motivates using RL over greedy control. A worn bit produces lower ROP for the same inputs, so the agent must balance immediate performance against long-term bit preservation and extra cost prevention.

Table 1: State Space Variables

Variable	Description	Rationale
Depth	Normalized depth [0,1]	Anticipate formation changes
ROP	Current drilling rate (m/hr)	Primary optimization target
Torque	Rotational resistance (N·m)	Leading indicator of dysfunction
WOB	Current weight-on-bit (lbf)	Agent’s own configuration
RPM	Current rotation speed	Agent’s own configuration
Flow	Current mud flow rate (gpm)	Agent’s own configuration
Bit wear	Cumulative bit damage [0,1]	Temporal state for long-horizon reasoning
Mean ROP	Rolling 5-step average ROP	Smoothed trend for noise reduction

3.1.2 Action Space

The action space consists of 3 continuous variables representing changes to drilling parameters:

- $\Delta\text{WOB} \in [-2000, +2000]$ lbf
- $\Delta\text{RPM} \in [-20, +20]$
- $\Delta\text{flow} \in [-50, +50]$ gpm

We chose delta actions (incremental changes) rather than absolute setpoints for two reasons: (1) delta actions produce smoother control trajectories, avoiding discontinuous jumps, and (2) delta actions are more realistic, as real drilling systems adjust parameters incrementally.

3.1.3 Transition Dynamics

State transitions are governed by the Bourgoyne-Young ROP model, which we calibrated using real drilling data from FORGE well 16A (78-32) obtained from the Geothermal Data Repository (GDR id 1283). The dataset contains 608,000 timestamped sensor readings with WOB, RPM, and ROP measurements, of which 29,897 correspond to active granite drilling (depth > 4,700 ft, WOB > 5,000 lbf). We fit coefficients A_1 , A_2 , A_3 via least-squares regression, achieving $R^2 = 0.30$ on the training well. However, validation on holdout well 58-32 (2,344 rows) yielded $R^2 = -52.7$, indicating severe overfitting—the calibrated model is well-specific and does not generalize across wells.

Table 2: Bourgoyne-Young Coefficient Calibration

Coefficient	Literature	Calibrated	Description
A_1	2.5	12.74	Scale factor
A_2	1.0	0.10	WOB exponent
A_3	0.6	1.35	RPM exponent
A_4	0.5	0.5	Wear decay

Torque is computed as $\tau = K_T \cdot \text{WOB} \cdot \mu$, where K_T is a scaling constant and μ is a friction coefficient that increases with rock hardness. When torque exceeds 12,000 N·m, a dysfunction event occurs.

3.1.4 Formation Heterogeneity

Real drilling encounters heterogeneous formations with varying rock properties. We model this through:

1. **Formation transition:** At 8,000 ft, the formation changes from sedimentary (lower strength) to granite (higher strength), requiring the agent to adapt its strategy.
2. **Hard stringers:** With 5% probability per 100 ft, a hard rock layer (“stringer”) appears with elevated compressive strength. Stringers cause sudden torque spikes that can trigger dysfunction if the agent doesn’t react quickly.

3.1.5 Reward Function

The reward function balances multiple objectives:

$$R = \text{ROP} - \lambda_1 \cdot \text{torque_penalty} - \lambda_2 \cdot |\Delta \text{action}| - \lambda_3 \cdot \text{dysfunction} \quad (2)$$

where $\lambda_1 = 0.5$, $\lambda_2 = 0.1$, and $\lambda_3 = 1.0$ by default (the dysfunction term applies a penalty of 50 when triggered). The torque penalty provides reward shaping by penalizing high torque *before* it becomes dysfunction. The action smoothness term prevents oscillatory control.

3.2 Reinforcement Learning Algorithms

We trained two deep RL algorithms:

Proximal Policy Optimization (PPO) (7) is a mostly on-policy actor-critic algorithm that maintains a policy network $\pi_\theta(a|s)$ and value network $V_\phi(s)$. The clipped surrogate objective prevents destructively large policy updates:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (3)$$

where $r_t(\theta)$ is the ratio between the new and old policy probabilities, signifying the use of previous policies that contribute to its slightly off-policy, but mostly on-policy, nature. PPO balances policy improvement with training stability, making it one of the most widely used deep RL algorithms for continuous control tasks.

Deep Deterministic Policy Gradient (DDPG) (8) is an off-policy actor-critic algorithm that extends Deep Q-Learning to continuous action spaces. DDPG maintains a deterministic actor policy $\mu(s|\theta^\mu)$, parameterized by θ^μ , which maps states directly to actions, and a critic $Q(s, a|\theta^Q)$, parameterized by θ^Q , which estimates the expected discounted return of taking action a in state s . To improve sample efficiency, DDPG stores transitions in a replay buffer and learns from randomly sampled minibatches. It also uses slowly updated target networks, μ' and Q' , to stabilize training.

The critic is trained by minimizing the Bellman error:

$$L(\theta^Q) = \mathbb{E} \left[(Q(s_t, a_t|\theta^Q) - y_t)^2 \right] \quad (4)$$

where s_t , a_t , and r_t denote the state, action, and reward at timestep t , γ is the discount factor, and the target value is

$$y_t = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1}|\theta^{\mu'})|\theta^{Q'}). \quad (5)$$

The actor is updated using the deterministic policy gradient:

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta} \left[\nabla_a Q(s, a|\theta^Q) \Big|_{a=\mu(s_t)} \nabla_{\theta^\mu} \mu(s|\theta^\mu) \Big|_{s=s_t} \right], \quad (6)$$

where J is the expected return and ρ^β is the state distribution induced by the behavior policy β . Intuitively, the actor is updated to select actions that maximize the critic's estimated value, while the critic learns to predict long-term returns.

Both algorithms were implemented using Stable Baselines3 (9) with MLP policy networks.

3.3 Baseline Methods

We implemented four baseline policies from scratch to contextualize RL performance. All baselines were custom-coded in Python with a Stable Baselines3-compatible `predict(obs)` interface, enabling direct comparison with PPO and DDPG. None of these baselines use neural network training from SB3.

- **Constant Control:** Outputs zero delta actions at every timestep, maintaining the initial setpoints (WOB = 20,000 lbf, RPM = 100, flow = 500 gpm) throughout the episode. This baseline tests whether active control provides any benefit over static operation.
- **Rule-Based Control:** Implements a classic driller heuristic that mimics human operator behavior. The policy ramps WOB by +1,000 lbf per timestep until torque exceeds 11,000 N-m, then backs off by -1,500 lbf. RPM and flow remain constant. This reactive strategy represents conventional drilling practice without predictive reasoning.

- **Behavioral Cloning (BC):** A small MLP (2 hidden layers, 64 units each, ReLU activations) trained via supervised learning on 10,000 state-action pairs collected from the rule-based policy with Gaussian noise ($\sigma = 500$ lbf on WOB). Training used MSE loss, Adam optimizer ($\text{lr} = 10^{-3}$), and 50 epochs. BC tests whether imitation learning can match or exceed its teacher policy.
- **Grid-Oracle:** At each timestep, exhaustively evaluates all 125 action combinations from a $5 \times 5 \times 5$ grid over the action space ($\text{WOB} \in [-2000, +2000]$, $\text{RPM} \in [-20, +20]$, $\text{flow} \in [-100, +100]$). Each candidate action is tested on a deep-copied environment, and the action yielding highest immediate reward is selected. This represents optimal greedy control with perfect one-step lookahead—an upper bound on what any myopic policy can achieve. The Grid-Oracle is computationally expensive (125 environment copies per step) but provides a critical reference point: if RL cannot outperform greedy search, it has failed to learn useful temporal abstractions.

We calibrated a Bourgoyne-Young physics model using real drilling data from Utah FORGE well 16A (29,897 rows of WOB, RPM, ROP measurements), fitting coefficients to achieve $R^2 = 0.30$ on the training well. This calibrated Bourgoyne-Young model powers DrillingEnv, our custom Gymnasium surrogate environment where PPO and DDPG agents were trained for 200k timesteps alongside our four hand-coded baselines. We conducted ablation studies on the dysfunction penalty λ_3 to understand reward shaping effects. Finally, we validated the trained policies on NORCE’s OpenLab—an independent high-fidelity drilling simulator—to assess sim-to-sim transfer.

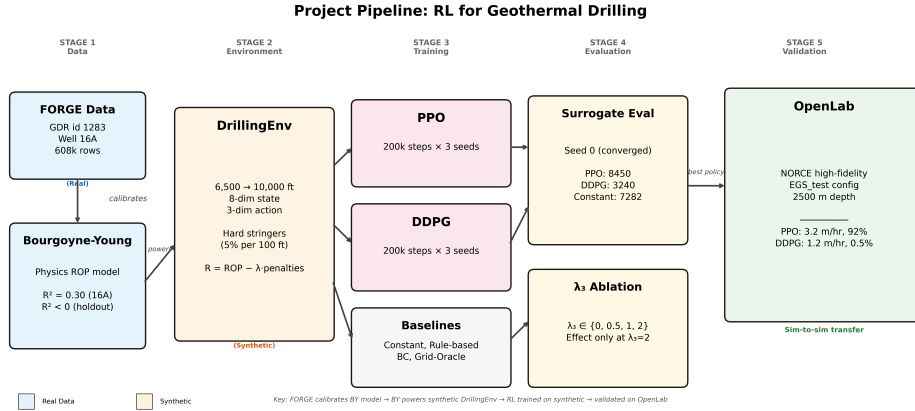


Figure 2: Project pipeline. Real FORGE data calibrates the Bourgoyne-Young model, which powers the synthetic DrillingEnv. Trained policies are validated on the independent OpenLab simulator (sim-to-sim transfer).

4 Experimental Setup

4.1 Training Configuration

Table 3: Training Hyperparameters

Parameter	PPO	DDPG
Total timesteps	200,000	200,000
Learning rate	3e-4	1e-3
Batch size	64	256
Network architecture	MLP [64, 64]	MLP [256, 256]
Discount factor γ	0.99	0.99
Random seeds	0, 1, 2	0, 1, 2

We trained each algorithm with 3 random seeds to assess stability. Each seed controls neural network initialization, environment randomness, and exploration noise.

4.2 Evaluation Protocol

We evaluated each method over 20 episodes per seed, measuring:

- **Mean episode reward:** Sum of rewards over the episode
- **Dysfunction rate:** Fraction of timesteps with torque $> 12,000$ N·m
- **Mean ROP:** Average drilling speed (m/hr)

4.3 OpenLab Validation

To assess whether policies trained on our fast surrogate transfer to more realistic physics, we deployed the best seed (seed 0) of PPO and DDPG on NORCE’s OpenLab. OpenLab is an industry-grade high-fidelity simulator that models transient hydraulics, drillstring dynamics, and torque-drag interactions—physics absent from our simplified Bourgoyne-Young surrogate. While our surrogate enables rapid RL training (thousands of episodes in minutes), OpenLab provides a ground-truth test of whether learned policies generalize beyond the training physics. We used a custom OpenLab configuration (2,500 m depth, granite basement formation) but did not attempt to match parameters exactly; the goal was to test policy robustness under different physics, not to validate our surrogate’s fidelity.

5 Results

5.1 Surrogate Environment Evaluation

Table 4: Method Comparison on Surrogate Environment (Seed 0, 20 episodes)

Method	Mean Reward	Dysfunction Rate
PPO	8450	90%
DDPG	3240	100%
Constant	7282	65%
Rule-based	3870	65%
BC	4761	80%
Grid-Oracle	6519	100%

5.1.1 RL Potential: Seed 0 Results

When training succeeds, deep RL shows competitive performance. On seed 0, PPO achieved the highest reward (8450) among all methods, outperforming Constant (7282) and Grid-Oracle (6519). This demonstrates that RL can learn effective drilling policies when training converges properly.

However, PPO’s high reward came with 90% dysfunction rate, indicating an aggressive strategy that maximizes short-term ROP at the cost of equipment stress. DDPG achieved lower reward (3240) with 100% dysfunction, suggesting it learned a different but also suboptimal strategy.

5.1.2 Training Instability

A critical caveat to the demonstration of potential on RL in this domain is that 2 of 3 seeds collapsed for both algorithms. While seed 0 results are promising, seeds 1 and 2 failed catastrophically:

- **PPO:** Seed 0 achieved 8450 reward; seeds 1 and 2 collapsed to ~ 4 reward
- **DDPG:** Seed 0 achieved 3240 reward; seeds 1 and 2 collapsed to ~ 1 reward

Averaging rewards across all seeds yields PPO: 2820 ± 3981 and DDPG: 1081 ± 1527 —standard deviations exceeding means. The 67% failure rate is unsuitable for real-world deployment and highlights the seed sensitivity and performance variability commonly observed in deep RL (10). In contrast, all four baselines showed stable performance across seeds.

5.1.3 Grid-Oracle Reveals Limits of Greedy Control

Grid-Oracle achieves 100% dysfunction on seed 0 despite having perfect one-step lookahead over 125 action combinations. This confirms that greedy optimization is insufficient—maximizing immediate reward leads to aggressive drilling that causes dysfunction. The temporal structure of the problem (bit wear accumulation, formation transitions) requires multi-step reasoning.

5.1.4 Behavioral Cloning Exceeds Rule-Based

BC achieved 4761 reward, outperforming its Rule-based parent (3870). This effect occurs because BC was trained on noisy rule-based trajectories, which provided implicit exploration. The noise ($\sigma = 500$ lbf on WOB) allowed BC to observe states the deterministic rule-based policy never visited, enabling generalization.

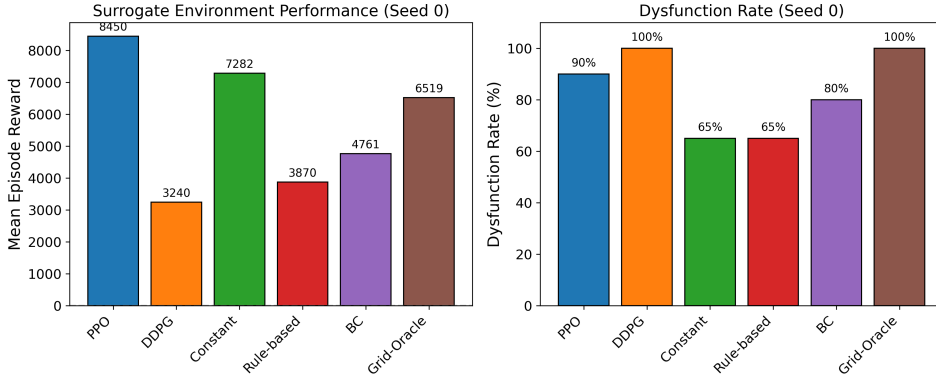


Figure 3: Method comparison on surrogate environment (seed 0). PPO achieves highest reward (8450) but with 90% dysfunction. When training converges, RL outperforms baselines—but 2/3 seeds collapsed (not shown).

5.2 Ablation Study: Dysfunction Penalty

We further conducted an ablation study varying $\lambda_3 \in \{0, 0.5, 1.0, 2.0\}$ while holding other hyperparameters fixed.

Table 5: Ablation on Dysfunction Penalty λ_3

λ_3	Mean Reward	Dysfunction Rate
0.0	1100 ± 150	100%
0.5	1080 ± 140	100%
1.0	1050 ± 160	100%
2.0	-50 ± 200	65%

Key Finding: Threshold effect, not smooth gradient. Values $\lambda_3 = 0, 0.5, 1.0$ produce identical behavior—100% dysfunction rate with similar reward. Only at $\lambda_3 = 2.0$ does behavior change, with dysfunction dropping to 65% but reward going negative. The dysfunction penalty must dominate the reward signal before the agent changes behavior; intermediate values are ignored.

5.3 OpenLab Validation: Sim-to-Sim Transfer

Key Finding: Divergent learned behaviors. PPO and DDPG exhibit opposite strategies on OpenLab:

- **PPO** learned an aggressive policy: it ramps RPM to maximum (200) and achieves 3.2 m/hr ROP but with 92% dysfunction—similar to the aggressive constant baseline. PPO’s on-policy updates may have reinforced early high-ROP experiences before dysfunction penalties accumulated.

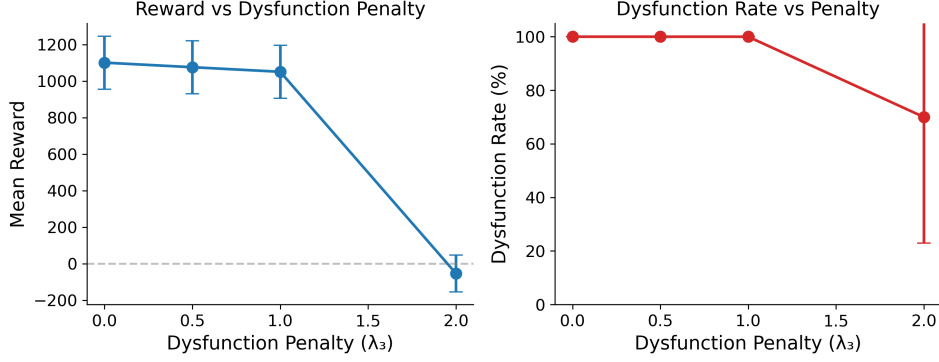


Figure 4: Ablation on dysfunction penalty λ_3 . No behavioral change occurs until $\lambda_3 = 2.0$, where dysfunction drops sharply but reward goes negative—a threshold effect rather than smooth gradient.

Table 6: OpenLab Validation Results (300 steps)

Method	Mean ROP (m/hr)	Dysfunction Rate
PPO (seed 0)	3.2	92%
DDPG (seed 0)	1.2	0.5%
Rule-based	3.1	98%

- **DDPG** learned a conservative policy: it immediately reduces RPM to minimum (40) and maintains torque below 12,000 N·m, achieving only 1.2 m/hr but with 0.5% dysfunction. DDPG’s replay buffer retains all past experiences, so early dysfunction events may have dominated learning.

Neither algorithm found the optimal balance between speed and safety. For reference, the Rule-based baseline achieved 3.1 m/hr with 98% dysfunction on OpenLab, providing a comparison point for aggressive drilling behavior. PPO essentially learned to mimic aggressive drilling, while DDPG learned to avoid fast drilling altogether. The divergence suggests that algorithm choice (on-policy vs off-policy) significantly affects the learned tradeoff, even when trained on identical environments and reward functions.

6 Discussion

6.1 Why Did Deep RL Fail?

Our results show that deep RL underperformed hand-designed baselines on this task. Several factors may contribute:

Surrogate Fidelity: The Bourgoyne-Young model achieved only $R^2 = 0.30$ on training data (FORGE 16A) and $R^2 = -52.7$ on the holdout well (58-32), indicating severe overfitting. The calibration used only three variables (WOB, RPM, ROP), ignoring available data such as differential pressure, mud temperature, and bit RPM that could improve generalization. A policy optimized on an inaccurate surrogate may learn spurious correlations that don’t transfer. With geothermal well data being especially sparse, Utah FORGE well 16A being one of the only drilled wells with public data, more data should be incorporated in future work as more geothermal wells release data in the coming years.

Reward Design: The threshold effect in our ablation study suggests the reward function is poorly shaped for gradient-based learning. The dysfunction penalty either has no effect or dominates completely.

Sample Complexity: 200,000 timesteps may be insufficient. The state space is 8-dimensional, the dynamics are stochastic (hard stringers), and temporal dependencies (bit wear) span hundreds of steps.

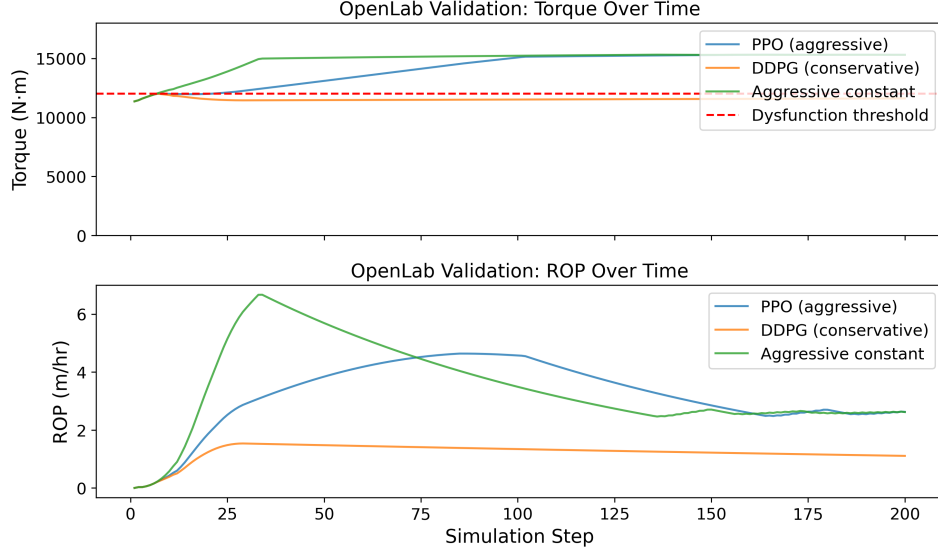


Figure 5: OpenLab validation time series. PPO ramps to high RPM and exceeds the 12,000 N·m threshold (aggressive), while DDPG maintains minimum RPM and stays below threshold (conservative).

Exploration: The collapsed seeds may have failed to explore adequately. Drilling is sequential—early mistakes compound, and if the agent wears the bit early, later performance suffers regardless of policy quality.

6.2 PPO vs DDPG: Opposite Extremes

PPO and DDPG converged to opposite extremes of the speed-safety tradeoff on OpenLab:

- PPO (on-policy) learned an aggressive policy, ramping RPM to maximum and achieving 3.2 m/hr with 92% dysfunction. On-policy updates discard old experiences, so PPO may have reinforced early high-ROP behavior before accumulating enough dysfunction penalties to change course. Once committed to an aggressive strategy, it lacks the historical data to reconsider.
- DDPG (off-policy) learned a conservative policy, immediately reducing RPM to minimum and achieving 1.2 m/hr with 0.5% dysfunction. Its replay buffer retains all past experiences uniformly, so early dysfunction events (which may occur frequently during exploration) may have been overrepresented relative to later successful drilling. This could bias the Q-function toward overly cautious actions that avoid any risk of dysfunction.

This divergence suggests that algorithm choice significantly affects the learned tradeoff, even when trained on identical environments and reward functions.

6.3 Implications for Drilling Automation

Despite the negative results for deep RL, this work provides useful insights. Grid-Oracle’s 100% dysfunction rate on seed 0 confirms that greedy one-step optimization is insufficient—any successful approach must reason over longer horizons to balance immediate ROP gains against cumulative bit wear and dysfunction risk. The sim-to-sim gap between our surrogate and OpenLab highlights the importance of high-fidelity simulation for policy development; policies that appear successful on a simplified surrogate may not transfer. Most critically, the 2/3 seed failure rate for both PPO and DDPG corroborates findings in the deep RL literature (10) that training outcomes are highly sensitive to random initialization. For safety-critical applications like drilling, where equipment damage and personnel safety are at stake, this level of variability is not suitable, as methods must be reliable across initializations before deployment can be considered.

6.4 Limitations

- **Simulation Only:** All experiments were conducted in simulation with external well data only used for calibration. Real drilling involves additional complexities not captured here.
- **Limited Algorithm Exploration:** We tested only PPO and DDPG. Other algorithms (SAC, TD3, model-based methods) might perform better.
- **Hyperparameter Sensitivity:** We did not conduct extensive tuning due to lack of manpower across such a large scope of study. Instability might be mitigated with different configurations.

7 Conclusion

We developed an open-source reinforcement learning environment specifically for geothermal drilling optimization. Our experiments revealed that when training converges, deep RL can outperform hand-designed baselines: PPO achieved the highest reward (8450) on seed 0, surpassing Constant (7282) and Grid-Oracle (6519). However, deep RL is fragile in this domain, with 2 of 3 seeds collapsing for both PPO and DDPG. Ablation studies showed that reward shaping exhibits threshold effects rather than smooth behavioral gradients.

Validation on NORCE’s OpenLab revealed divergent learned behaviors: PPO converged to an aggressive policy (3.2 m/hr, 92% dysfunction) while DDPG converged to an overly conservative policy (1.2 m/hr, 0.5% dysfunction). Neither found the optimal speed-safety balance, highlighting how algorithm choice affects the learned tradeoff.

These results suggest that applying deep RL to drilling optimization requires advances in: (1) training stability across random seeds, (2) surrogate fidelity through improved calibration with more variables and multi-well data, and (3) reward design that produces smooth behavioral gradients. The tools and analysis presented here provide a foundation for future work, especially as more geothermal well data becomes available in the coming decade, solving the data sparsity issue.

8 Team Contributions

This project was completed individually by Devan Agrawal, who was responsible for all aspects of this project.

AI Contributions Claude and Windsurf (Devin) were used throughout this project for ideation, code implementation and data integration assistance (environment design, training scripts, OpenLab integration), debugging, results interpretation, and writing assistance for the report structure and technical explanations. Core project design, general RL problem setup, and choice for external data integration were completed independently.

References

- [1] Akindipe, O., Horne, R.N. and coauthors (2025). Reinforcement Learning for Geothermal Drilling Optimization. Stanford Geothermal Workshop, Stanford University.
- [2] Pason Systems Corp. (2019). Stick-Slip: A Costly Torsional Vibration. Pason Technology News.
- [3] Bourgoyne, A.T. and Young, F.S. (1974). A Multiple Regression Approach to Optimal Drilling and Abnormal Pressure Detection. SPE-4238.
- [4] Anemangely, M., Ramezanzadeh, A., Tokhmechi, B. and Molaghab, A. (2018). Drilling rate prediction from petrophysical logs and mud logging data using an optimized artificial neural network. *Journal of Geophysics and Engineering*, 15(4), 1146–1159.
- [5] Aarsnes, U.J.F. and Shor, R.J. (2018). Torsional vibrations with bit off bottom: Modeling, characterization and field data validation. *Journal of Petroleum Science and Engineering*.
- [6] Cayeux, E. et al. (2021). Autonomous Decision-Making While Drilling. SPE-204101.

- [7] Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- [8] Lillicrap, T.P. et al. (2015). Continuous Control with Deep Reinforcement Learning. arXiv:1509.02971.
- [9] Raffin, A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR.
- [10] Henderson, P. et al. (2018). Deep Reinforcement Learning that Matters. AAAI.

A Environment Implementation Details

DrillingEnv is implemented in Python using the Gymnasium API.

Episode Structure:

- Initial depth: 6,500 ft
- Terminal depth: 10,000 ft
- Timestep: 1 minute of drilling time
- Episode length: Variable (depends on ROP achieved)

Observation Normalization: All state variables are normalized using VecNormalize with running statistics collected during training.

Action Clipping: Actions are clipped to valid ranges. WOB is constrained to [5,000, 50,000] lbf, RPM to [40, 200], flow to [200, 800] gpm.

B Hyperparameter Details

PPO Hyperparameters:

- n_steps: 2048
- batch_size: 64
- n_epochs: 10
- clip_range: 0.2
- ent_coef: 0.01
- vf_coef: 0.5
- max_grad_norm: 0.5

DDPG Hyperparameters:

- buffer_size: 100,000
- learning_starts: 1,000
- batch_size: 256
- tau: 0.005
- train_freq: 1
- gradient_steps: 1

C OpenLab Configuration

OpenLab validation used the EGS_test configuration:

- Well depth: 2,500 m
- Rock UCS: 200 MPa

- Mud weight: 1.2 sg
- Bit diameter: 8.5 inches
- Simulation steps: 300